

JavaScript

50 Exercícios — Fundamentos

Strings · Números · Arrays · Objetos · Funções · Controle de Fluxo

■ 25 Fáceis ■ 25 Médios

- Resolva cada exercício antes de ver a solução proposta.
- Os exemplos mostram uma possível solução — existem muitas formas corretas.
- Use as dicas (■) quando estiver travada, não antes de tentar.
- Os exercícios dentro de cada nível são ordenados por complexidade crescente.

Índice

■ Fáceis (exercícios 1–25)

1. trim() e length
2. toUpperCase e toLowerCase
3. includes e startsWith
4. replaceAll e split
5. slice de string
6. padStart para formatação
7. parseInt e parseFloat
8. Math.round, floor e ceil
9. Math.max, min e sqrt
10. toFixed — casas decimais
11. Math.random — número aleatório
12. Criar e acessar array
13. push, pop, shift e unshift
14. forEach com índice
15. includes em array
16. Criar e acessar objeto
17. Object.keys e Object.values
18. Método dentro de objeto
19. Destructuring de objeto
20. Destructuring de array
21. Função declarada e arrow function

- 22. Parâmetro padrão
- 23. Rest parameters
- 24. for...of em array
- 25. if / else com operador %

■ Médios (exercícios 26–50)

- 26. Inverter uma string
- 27. Contar ocorrências de um caractere
- 28. Capitalizar primeira letra
- 29. Verificar palíndromo
- 30. Contar palavras numa frase
- 31. Somar com reduce
- 32. map — transformar array
- 33. filter — filtrar objetos
- 34. Remover duplicatas com Set
- 35. sort — ordenar strings e números
- 36. find e findIndex
- 37. every e some
- 38. Array.from — criar sequência
- 39. flat e flatMap
- 40. Encadear filter + map + reduce
- 41. Spread em objetos — clone e merge
- 42. Optional chaining e Nullish
- 43. Object.entries e for...of
- 44. Contar frequência de itens
- 45. Closure — contador privado
- 46. Função que retorna função
- 47. for com break — busca linear
- 48. for com continue
- 49. While com validação
- 50. Pipeline: filtrar, transformar, exibir

■ Exercícios Fáceis | Exercícios 1–25 | Strings · Números · Arrays · Objetos · Funções · Controle de Fluxo

1 trim() e length ■ Fácil Strings

Dada a string ' olá, mundo! ', remova os espaços das bordas e exiba o comprimento do resultado.

```
const s = ' olá, mundo! ';  
const limpa = s.trim();  
console.log(limpa);           // 'olá, mundo!'  
console.log(limpa.length);    // 12
```

■ trim() remove apenas espaços do início e do fim, não do meio.

■ string · trim · length

2 toUpperCase e toLowerCase ■ Fácil Strings

Dada a string 'JavaScript é Incrível', exiba-a toda em maiúsculas e toda em minúsculas.

```
const s = 'JavaScript é Incrível';
console.log(s.toUpperCase()); // JAVASCRIPT É INCRÍVEL
console.log(s.toLowerCase()); // javascript é incrível
```

■ Estes métodos não modificam a string original — retornam uma nova.

■ [string](#) · [toUpperCase](#) · [toLowerCase](#)

3 includes e startsWith ■ Fácil Strings

Verifique se a string 'Programação em JavaScript' contém 'Script' e se começa com 'Prog'.

```
const frase = 'Programação em JavaScript';
console.log(frase.includes('Script')); // true
console.log(frase.startsWith('Prog')); // true
console.log(frase.startsWith('Java')); // false
```

■ `includes()` e `startsWith()` são case-sensitive — 'script' seria false.

■ [string](#) · [includes](#) · [startsWith](#)

4 replaceAll e split ■ Fácil Strings

Substitua todos os espaços de 'arroz feijão milho trigo' por vírgulas. Depois divida a string em array de palavras.

```
const s = 'arroz feijão milho trigo';
console.log(s.replaceAll(' ', ','));
// 'arroz,feijão,milho,trigo'
console.log(s.split(' '));
// ['arroz','feijão','milho','trigo']
```

■ `split(' ')` divide a string em cada espaço encontrado.

■ [string](#) · [replaceAll](#) · [split](#)

5 slice de string ■ Fácil Strings

Dada a string 'Desenvolvimento', extraia 'Desen' (primeiros 5 chars) e 'mento' (últimos 5 chars).

```
const s = 'Desenvolvimento';
console.log(s.slice(0, 5)); // 'Desen'
console.log(s.slice(-5)); // 'mento'
console.log(s.length); // 15
```

■ Índice negativo em `slice()` conta a partir do final da string.

■ [string](#) · [slice](#)

6 padStart para formatação ■ Fácil Strings

Dado um número de 1 a 999, formate-o como string de 4 dígitos com zeros à esquerda (ex: 42 → '0042').

```
const n = 42;
const formatado = String(n).padStart(4, '0');
console.log(formatado); // '0042'

console.log(String(7).padStart(4, '0')); // '0007'
console.log(String(999).padStart(4, '0')); // '0999'
```

■ `padStart(tamanho, char)` preenche à esquerda até atingir o tamanho.

■ [string](#) · [padStart](#)

7 parseInt e parseFloat ■ Fácil Números

Converta '100', '3.99' e '50px' para número. Teste também isNaN() com uma string inválida.

```
console.log(parseInt('100')); // 100
console.log(parseFloat('3.99')); // 3.99
console.log(parseInt('50px')); // 50
console.log(isNaN('abc')); // true
console.log(isNaN('42')); // false
```

■ parseInt pára quando encontra um caractere não numérico.

■ number · parseInt · parseFloat · isNaN

8 Math.round, floor e ceil ■ Fácil Números

Dado o número 7.6, aplique Math.round(), Math.floor() e Math.ceil(). Repita com 7.2.

```
console.log(Math.round(7.6)); // 8 (arredonda normal)
console.log(Math.floor(7.6)); // 7 (arredonda para baixo)
console.log(Math.ceil(7.6)); // 8 (arredonda para cima)

console.log(Math.round(7.2)); // 7
console.log(Math.floor(7.2)); // 7
console.log(Math.ceil(7.2)); // 8
```

■ floor() sempre arredonda para baixo, ceil() sempre para cima.

■ Math · number

9 Math.max, min e sqrt ■ Fácil Números

Use Math para encontrar o maior e o menor entre 4 números, e calcular a raiz quadrada de 144.

```
console.log(Math.max(3, 8, 1, 6)); // 8
console.log(Math.min(3, 8, 1, 6)); // 1
console.log(Math.sqrt(144)); // 12
console.log(Math.abs(-25)); // 25
```

■ Math.abs() retorna o valor absoluto (sem sinal negativo).

■ Math · number

10 toFixed — casas decimais ■ Fácil Números

Dado o resultado da divisão 10 / 3, exiba-o com 2 e com 4 casas decimais.

```
const resultado = 10 / 3;
console.log(resultado.toFixed(2)); // '3.33' (string!)
console.log(resultado.toFixed(4)); // '3.3333'
console.log(typeof resultado.toFixed(2)); // 'string'
```

■ toFixed() retorna uma string — use Number() se precisar de número.

■ number · toFixed

11 Math.random — número aleatório ■ Fácil Números

Gere um número inteiro aleatório entre 1 e 10 (inclusive) e entre 1 e 100.

```
// Fórmula: Math.floor(Math.random() * (max - min + 1)) + min
const dado = Math.floor(Math.random() * 10) + 1;
console.log(dado); // 1 a 10

const centena = Math.floor(Math.random() * 100) + 1;
console.log(centena); // 1 a 100
```

■ `Math.random()` retorna `[0, 1)` — nunca chega exatamente a 1.

■ `Math` · `number` · `Math.random`

12 Criar e acessar array ■ Fácil Arrays

Crie um array com 5 cores. Acesse a primeira, a última e o total de elementos.

```
const cores = ['vermelho', 'verde', 'azul', 'amarelo', 'roxo'];
console.log(cores[0]); // 'vermelho'
console.log(cores[cores.length - 1]); // 'roxo'
console.log(cores.length); // 5
```

■ O índice do último elemento é sempre `length - 1`.

■ `array` · `índice` · `length`

13 push, pop, shift e unshift ■ Fácil Arrays

Dado um array `['b','c','d']`, adicione 'a' no início, 'e' no final, remova do início e do final.

```
const arr = ['b', 'c', 'd'];
arr.unshift('a'); // adiciona no início
arr.push('e'); // adiciona no final
console.log(arr); // ['a', 'b', 'c', 'd', 'e']
arr.shift(); // remove do início
arr.pop(); // remove do final
console.log(arr); // ['b', 'c', 'd']
```

■ `push/pop` agem no final; `shift/unshift` agem no início.

■ `array` · `push` · `pop` · `shift` · `unshift`

14 forEach com índice ■ Fácil Arrays

Dado um array de linguagens, use `forEach` para exibir cada uma no formato '1. JavaScript'.

```
const langs = ['JavaScript', 'Python', 'Java', 'C#'];
langs.forEach((lang, i) => {
  console.log(`${i + 1}. ${lang}`);
});
// 1. JavaScript
// 2. Python ...
```

■ `forEach` recebe (`elemento`, `índice`, `array`) — o índice começa em 0.

■ `array` · `forEach` · `arrow function`

15 includes em array ■ Fácil Arrays

Dado um array de números primos, verifique se 7 e 10 estão no array.

```
const primos = [2, 3, 5, 7, 11, 13];
console.log(primos.includes(7)); // true
console.log(primos.includes(10)); // false
```

■ *includes() usa comparação estrita (===) internamente.*

■ [array](#) · [includes](#)

16 Criar e acessar objeto ■ Fácil Objetos

Crie um objeto representando um carro com marca, modelo e ano. Acesse via notação ponto e colchetes.

```
const carro = { marca: 'Toyota', modelo: 'Corolla', ano: 2022 };
console.log(carro.marca); // 'Toyota'
console.log(carro['modelo']); // 'Corolla'
console.log(carro.ano); // 2022
```

■ *Notação colchetes é útil quando a chave é dinâmica ou tem espaços.*

■ [objeto](#) · [propriedades](#)

17 Object.keys e Object.values ■ Fácil Objetos

Dado um objeto aluno com nome, turma e nota, liste suas chaves e seus valores.

```
const aluno = { nome: 'Ana', turma: 'A', nota: 9.5 };
console.log(Object.keys(aluno));
// ['nome', 'turma', 'nota']
console.log(Object.values(aluno));
// ['Ana', 'A', 9.5]
```

■ *Object.entries() retorna pares [chave, valor] — combina os dois.*

■ [objeto](#) · [Object.keys](#) · [Object.values](#)

18 Método dentro de objeto ■ Fácil Objetos

Crie um objeto circulo com raio. Adicione métodos area() e perimetro() que usam this.

```
const circulo = {
  raio: 5,
  area() {
    return Math.PI * this.raio ** 2;
  },
  perimetro() {
    return 2 * Math.PI * this.raio;
  }
};
console.log(circulo.area().toFixed(2)); // 78.54
console.log(circulo.perimetro().toFixed(2)); // 31.42
```

■ *Dentro de métodos, this refere-se ao próprio objeto.*

■ [objeto](#) · [método](#) · [this](#) · [Math](#)

19 Destructuring de objeto ■ Fácil Objetos

Dado um objeto filme com titulo, diretor e ano, extraia as três propriedades com destructuring.

```
const filme = { titulo: 'Matrix', diretor: 'Wachowski', ano: 1999 };
const { titulo, diretor, ano } = filme;
console.log(titulo); // 'Matrix'
console.log(diretor); // 'Wachowski'
console.log(ano);    // 1999
```

■ Você pode renomear ao destruturar: `const { titulo: t } = filme;`

■ [objeto](#) · [destructuring](#)

20 Destructuring de array ■ Fácil Arrays

Dado o array [100, 200, 300, 400, 500], extraia o 1.º em 'primeiro', o 3.º em 'terceiro'. Ignore os outros.

```
const [primeiro, , terceiro] = [100, 200, 300, 400, 500];
console.log(primeiro); // 100
console.log(terceiro); // 300
```

■ Use vírgulas extras para pular posições no destructuring de array.

■ [array](#) · [destructuring](#)

21 Função declarada e arrow function ■ Fácil Funções

Crie a função potencia(base, exp) que calcula base elevado a exp. Escreva nas duas sintaxes.

```
// Declaração clássica
function potencia(base, exp) {
  return base ** exp;
}

// Arrow function
const potenciaArrow = (base, exp) => base ** exp;

console.log(potencia(2, 8));    // 256
console.log(potenciaArrow(3, 3)); // 27
```

■ Com um único return, a arrow function pode omitir as chaves e a palavra return.

■ [function](#) · [arrow function](#) · [return](#)

22 Parâmetro padrão ■ Fácil Funções

Crie a função cumprimentar(nome, hora = 'manhã') que exibe 'Bom dia de [hora], [nome]!'.

```
const cumprimentar = (nome, hora = 'manhã') => {
  console.log(`Bom dia de ${hora}, ${nome}!`);
};

cumprimentar('Gi');           // Bom dia de manhã, Gi!
cumprimentar('Gi', 'tarde'); // Bom dia de tarde, Gi!
```

■ O parâmetro padrão só é ativado quando o argumento é undefined.

■ [funções](#) · [parâmetro padrão](#) · [arrow function](#)

23 Rest parameters ■ Fácil Funções

Crie a função `media(...notas)` que calcula a média de qualquer quantidade de notas.

```
const media = (...notas) => {
  const soma = notas.reduce((acc, n) => acc + n, 0);
  return soma / notas.length;
};

console.log(media(8, 9, 10)); // 9
console.log(media(6, 7, 5, 8)); // 6.5
```

■ *...rest coleta todos os argumentos num array — use `.length` para contá-los.*

■ [funções](#) · [rest parameters](#) · [reduce](#)

24 for...of em array ■ Fácil Controle de Fluxo

Dado um array de temperaturas em Celsius, use `for...of` para exibir cada uma convertida em Fahrenheit.

```
const celsius = [0, 20, 37, 100];

for (const c of celsius) {
  const f = c * 9/5 + 32;
  console.log(`${c}°C = ${f}°F`);
}

// 0°C = 32°F
// 20°C = 68°F ...
```

■ *for...of é mais limpo que o for clássico quando não precisa do índice.*

■ [for...of](#) · [array](#) · [template literals](#)

25 if / else com operador % ■ Fácil Controle de Fluxo

Dado um array de números de 1 a 8, use `for...of` e `if/else` para exibir 'Par' ou 'Ímpar' para cada.

```
const nums = [1, 2, 3, 4, 5, 6, 7, 8];

for (const n of nums) {
  if (n % 2 === 0) {
    console.log(`${n} → Par`);
  } else {
    console.log(`${n} → Ímpar`);
  }
}
```

■ *`n % 2 === 0` verifica se o resto da divisão por 2 é zero.*

■ [for...of](#) · [if](#) · [else](#) · [operador %](#)

■ Exercícios Médios | Exercícios 26–50 | Strings · Arrays · Objetos · Funções · Controle de Fluxo

26 Inverter uma string ■ Médio Strings

Crie uma função que recebe uma string e retorna ela de trás para frente. Ex: 'amor' → 'roma'.

```
const inverter = str => [...str].reverse().join('');

console.log(inverter('amor')); // 'roma'
console.log(inverter('JavaScript')); // 'tpircSavaJ'
```

■ Spread de string cria um array de caracteres que pode ser invertido com reverse().

■ string · spread · array · reverse · join

27 Contar ocorrências de um caractere ■ Médio Strings

Crie uma função que conta quantas vezes um caractere específico aparece numa string.

```
const contarChar = (str, char) =>
  [...str].filter(c => c === char).length;

console.log(contarChar('banana', 'a')); // 3
console.log(contarChar('abacaxi', 'a')); // 3
```

■ Spread de string + filter é uma forma elegante de contar caracteres.

■ string · spread · filter · array

28 Capitalizar primeira letra ■ Médio Strings

Crie uma função que transforma a primeira letra de uma string em maiúscula e o resto em minúsculas.

```
const capitalizar = str =>
  str.charAt(0).toUpperCase() + str.slice(1).toLowerCase();

console.log(capitalizar('javascript')); // 'Javascript'
console.log(capitalizar('HELLO')); // 'Hello'
```

■ charAt(0) acessa o 1.º caractere; slice(1) pega do 2.º em diante.

■ string · charAt · toUpperCase · toLowerCase · slice

29 Verificar palíndromo ■ Médio Strings

Crie uma função que verifica se uma palavra é palíndromo (lê-se igual de frente e de trás), ignorando maiúsculas.

```
const ehPalindromo = str => {
  const limpa = str.toLowerCase();
  return limpa === [...limpa].reverse().join('');
};

console.log(ehPalindromo('Arara')); // true
console.log(ehPalindromo('radar')); // true
console.log(ehPalindromo('hello')); // false
```

■ toLowerCase() antes da comparação garante case-insensitive.

■ string · spread · reverse · join · toLowerCase

30 Contar palavras numa frase ■ Médio Strings

Crie uma função que recebe uma frase e retorna o número de palavras. Lide com espaços extras.

```
const contarPalavras = frase =>
  frase.trim().split(/\s+/).length;

console.log(contarPalavras('Olá mundo')); // 2
console.log(contarPalavras(' três palavras aqui ')); // 3
```

■ A regex `/\s+/` divide por qualquer quantidade de espaços consecutivos.

■ `string` · `trim` · `split` · `regex`

31 Somar com reduce ■ Médio Arrays

Dado um array de preços, use `reduce()` para calcular o total a pagar.

```
const precos = [12.5, 9.9, 34.0, 4.99];
const total = precos.reduce((acc, p) => acc + p, 0);
console.log(total.toFixed(2)); // '61.39'
```

■ O segundo argumento de `reduce()` é o valor inicial do acumulador.

■ `array` · `reduce` · `toFixed`

32 map — transformar array ■ Médio Arrays

Dado um array de nomes em minúsculas, use `map()` para criar um novo array com cada nome capitalizado.

```
const nomes = ['ana', 'bruno', 'carla', 'davi'];
const capitalizados = nomes.map(
  n => n.charAt(0).toUpperCase() + n.slice(1)
);
console.log(capitalizados);
// ['Ana', 'Bruno', 'Carla', 'Davi']
```

■ `map()` retorna um novo array sem modificar o original.

■ `array` · `map` · `string` · `charAt` · `slice`

33 filter — filtrar objetos ■ Médio Arrays

Dado um array de produtos com preço, filtre apenas os que custam menos de 50.

```
const produtos = [
  { nome: 'Caneta', preco: 3 },
  { nome: 'Livro', preco: 45 },
  { nome: 'Tablet', preco: 350 },
  { nome: 'Borracha', preco: 2 },
];
const baratos = produtos.filter(p => p.preco < 50);
console.log(baratos.map(p => p.nome));
// ['Caneta', 'Livro', 'Borracha']
```

■ `filter()` retorna um novo array com os elementos que passam no teste.

■ `array` · `filter` · `objeto` · `map`

34 Remover duplicatas com Set ■ Médio Arrays

Dado um array com nomes repetidos, crie um novo array com apenas os nomes únicos.

```
const nomes = ['Ana', 'Bia', 'Ana', 'Carlos', 'Bia', 'Ana'];
const unicos = [...new Set(nomes)];
console.log(unicos); // ['Ana', 'Bia', 'Carlos']
```

■ Set aceita apenas valores únicos; spread converte de volta para array.

■ array · Set · spread

35 sort — ordenar strings e números ■ Médio Arrays

Ordene um array de nomes alfabeticamente e um array de números do maior para o menor.

```
const nomes = ['Zara', 'Ana', 'Pedro', 'Bia'];
nomes.sort();
console.log(nomes); // ['Ana', 'Bia', 'Pedro', 'Zara']

const nums = [42, 5, 100, 17, 3];
nums.sort((a, b) => b - a);
console.log(nums); // [100, 42, 17, 5, 3]
```

■ Sem comparador, sort() ordena como string — para números use (a,b) => a-b.

■ array · sort · localeCompare

36 find e findIndex ■ Médio Arrays

Dado um array de alunos com nome e nota, encontre o primeiro com nota abaixo de 5 e seu índice.

```
const alunos = [
  { nome: 'Ana', nota: 8 },
  { nome: 'Bruno', nota: 4.5 },
  { nome: 'Carla', nota: 3 },
];
const reprovado = alunos.find(a => a.nota < 5);
const idx = alunos.findIndex(a => a.nota < 5);
console.log(reprovado.nome); // 'Bruno'
console.log(idx);           // 1
```

■ find() retorna o elemento; findIndex() retorna a posição — ambos param no primeiro match.

■ array · find · findIndex · objeto

37 every e some ■ Médio Arrays

Dado um array de temperaturas, verifique se todas estão acima de 0 e se alguma passa de 35.

```
const temps = [12, 28, -3, 36, 15];
console.log(temps.every(t => t > 0)); // false (há -3)
console.log(temps.some(t => t > 35)); // true (há 36)
```

■ every() para no primeiro false; some() para no primeiro true.

■ array · every · some

38 Array.from — criar sequência ■ Médio Arrays

Use Array.from() para criar um array com os números pares de 2 a 20.

```
const pares = Array.from(
  { length: 10 },
  (_, i) => (i + 1) * 2
);
console.log(pares);
// [2,4,6,8,10,12,14,16,18,20]
```

■ Array.from aceita um segundo argumento de mapeamento: (valorIgnorado, índice).

■ array · Array.from

39 flat e flatMap ■ Médio Arrays

Dado [[1,2],[3,4],[5,6]], achate o array. Depois, dado [1,2,3], crie ['1 pts','2 pts','3 pts'] com flatMap.

```
console.log([[1,2],[3,4],[5,6]].flat());
// [1,2,3,4,5,6]

const resultado = [1, 2, 3].flatMap(n => [`${n} pts`]);
console.log(resultado);
// ['1 pts', '2 pts', '3 pts']
```

■ flatMap() faz map() + flat(1) em uma só operação.

■ array · flat · flatMap

40 Encadear filter + map + reduce ■ Médio Arrays

Dado um array de 1 a 10, filtre apenas os pares, eleve ao quadrado e some tudo.

```
const resultado = [1,2,3,4,5,6,7,8,9,10]
  .filter(n => n % 2 === 0)
  .map(n => n ** 2)
  .reduce((acc, n) => acc + n, 0);

console.log(resultado); // 220
```

■ Encadear métodos de array é a essência da programação funcional em JS.

■ array · filter · map · reduce

41 Spread em objetos — clone e merge ■ Médio Objetos

Clone um objeto config1. Depois, crie um novo objeto mesclando config1 e config2 (config2 tem prioridade).

```
const cfg1 = { tema: 'dark', fonte: 14, idioma: 'pt' };
const cfg2 = { fonte: 16, notif: true };

const clone = { ...cfg1 };
const merged = { ...cfg1, ...cfg2 };

console.log(clone);
// { tema:'dark', fonte:14, idioma:'pt' }
console.log(merged);
// { tema:'dark', fonte:16, idioma:'pt', notif:true }
```

■ Quando chaves se repetem no spread, a última prevalece.

■ objeto · spread · clone · merge

42 Optional chaining (?.) e Nullish (??) ■ Médio Objetos

Acesse cidade de dois objetos usuário (um sem endereço) com ?. Use ?? para valor padrão.

```
const u1 = { nome: 'Ana', endereco: { cidade: 'Porto' } };
const u2 = { nome: 'Bob' };

console.log(u1?.endereco?.cidade ?? 'Desconhecida'); // 'Porto'
console.log(u2?.endereco?.cidade ?? 'Desconhecida'); // 'Desconhecida'
```

■ ?. retorna undefined sem lançar erro; ?? usa o lado direito só se for null/undefined.

■ [objeto](#) · [optional chaining](#) · [nullish coalescing](#)

43 Object.entries e for...of ■ Médio Objetos

Dado um objeto com notas de alunos, use Object.entries() com for...of para exibir cada aluno e média.

```
const notas = { Ana: [8,9,10], Bob: [5,6,5], Cia: [7,8,9] };

for (const [aluno, lista] of Object.entries(notas)) {
  const media = lista.reduce((a,b)=>a+b,0) / lista.length;
  console.log(`${aluno}: ${media.toFixed(1)}`);
}
// Ana: 9.0 | Bob: 5.3 | Cia: 8.0
```

■ Object.entries() transforma o objeto num array de [chave, valor].

■ [objeto](#) · [Object.entries](#) · [for...of](#) · [destructuring](#) · [reduce](#)

44 Contar frequência de itens ■ Médio Objetos

Dado um array de frutas com repetições, use reduce() para contar quantas vezes cada uma aparece.

```
const frutas = ['maçã', 'banana', 'maçã', 'uva', 'banana', 'maçã'];
const freq = frutas.reduce((acc, f) => {
  acc[f] = (acc[f] || 0) + 1;
  return acc;
}, {});
console.log(freq);
// { maçã:3, banana:2, uva:1 }
```

■ Inicialize cada chave com 0 na primeira ocorrência antes de incrementar.

■ [array](#) · [reduce](#) · [objeto](#)

45 Closure — contador privado ■ Médio Funções

Use closure para criar um contador com estado interno privado, expondo incrementar() e valor().

```
const criarContador = (inicio = 0) => {
  let count = inicio;
  return {
    incrementar: () => ++count,
    decrementar: () => --count,
    valor: () => count,
  };
};

const c = criarContador(5);
c.incrementar();
c.incrementar();
console.log(c.valor()); // 7
```

■ O estado `count` é privado — só acessível pelas funções retornadas.

■ closure · estado privado · objeto · arrow function

46 Função que retorna função ■ Médio Funções

Crie saudador(saudacao) que retorna uma função recebendo nome e exibindo '[saudacao], [nome]!'.

```
const saudador = saudacao => nome =>
  console.log(`${saudacao}, ${nome}!`);

const bomDia = saudador('Bom dia');
const boaTarde = saudador('Boa tarde');

bomDia('Ana');    // Bom dia, Ana!
boaTarde('Gi');   // Boa tarde, Gi!
```

■ Funções que retornam funções são a base de closures e currying.

■ closure · higher-order function · arrow function

47 for com break — busca linear ■ Médio Controle de Fluxo

Dado um array de palavras, use for...of com break para encontrar a primeira que começa com 'J'.

```
const palavras = ['Python', 'Java', 'JavaScript', 'Julia', 'Ruby'];

for (const p of palavras) {
  if (p.startsWith('J')) {
    console.log('Encontrei:', p); // 'Java'
    break;
  }
}
```

■ break encerra o loop imediatamente ao encontrar o resultado.

■ for...of · break · string · startsWith

48 for com continue ■ Médio Controle de Fluxo

Dado um array de números, use for...of com continue para imprimir apenas os múltiplos de 3.

```
const nums = [1,2,3,4,5,6,7,8,9,10,11,12];

for (const n of nums) {
  if (n % 3 !== 0) continue;
  console.log(n);
}
// 3, 6, 9, 12
```

■ continue pula para a próxima iteração sem executar o restante do bloco.

■ for...of · continue · operador %

49 While com validação ■ Médio Controle de Fluxo

Simule um caixa eletrônico: use while para debitar 100 do saldo enquanto houver saldo suficiente.

```
let saldo = 550;
let saques = 0;

while (saldo >= 100) {
  saldo -= 100;
  saques++;
}

console.log(`Saques: ${saques}`); // 5
console.log(`Saldo restante: ${saldo}`); // 50
```

■ Garanta que a condição do while eventualmente se torne falsa para evitar loop infinito.

■ while · loop · variáveis

50 Pipeline: filtrar, transformar, exibir ■ Médio Controle de Fluxo

Dado um array de alunos com nome e nota, filtre aprovados (>=6), ordene por nota decrescente e exiba.

```
const alunos = [
  {nome: 'Ana', nota: 9}, {nome: 'Bob', nota: 4},
  {nome: 'Cia', nota: 7}, {nome: 'Dan', nota: 6},
  {nome: 'Eva', nota: 3},
];

alunos
  .filter(a => a.nota >= 6)
  .sort((a,b) => b.nota - a.nota)
  .forEach((a,i) =>
    console.log(`${i+1}. ${a.nome} - ${a.nota}`)
  );
// 1. Ana - 9 | 2. Cia - 7 | 3. Dan - 6
```

■ Encadear filter → sort → forEach é um padrão muito comum em JS.

■ array · filter · sort · forEach · objeto
